

DAG-Based Execution Engine ”ninja”: Functional Architecture, Code Specification, and Rust Integration

Atsushi Shimmura (kyo38)

2026 年 7 月 27 日

Contents

本ドキュメントの前提と範囲

本ドキュメントは、リポジトリ `kyo38/ninja` (<https://github.com/kyo38/ninja>) における ****DAG (Directed Acyclic Graph: 有向非巡回グラフ) **** をベースとしたタスク依存関係解析エンジンの各機能モジュール別コード解説、および ****Rust からの C-FFI 経由での呼び出し手法**** に全振りして記述した技術仕様書である。

- 解説の対象: DAG のノード・エッジ管理、サイクル検知アルゴリズム、Kahn のアルゴリズムを用いたトポロジカルソート実行エンジン、C-FFI エクスポート層、および Rust からの安全なバインディング呼出。
- 環境・プロトコルの割愛: SCION プロトコル、特定ルーター/FW 機器構成、ネットワークポロジ等の環境記述はすべて割愛する。
- 動作検証に関する注意事項: ****本プログラムおよび提示するコード構造の Linux 環境における実行動作は未検証 (Unverified) である。***本書は計算モデル・データ構造・言語間連携インターフェースの設計解説として記述されている。

1 `ninja` エンジンの機能ブロック概要

`ninja` のコアエンジンは、依存関係を持つ一連のタスクを正しく順序付け、非同期かつ安全に実行するために以下の 4 つの機能モジュールで構成されている。

1. **グラフ構築モジュール (Graph Builder):** 頂点 (Task Node) と有向辺 (Dependency Edge) の動的登録、隣接リストおよび入次数 (In-Degree) の管理。
2. **依存関係検証・サイクル検知モジュール (Cycle Detector):** DFS (深さ優先探索) を用いた三色塗りつぶし法による非巡回性 (Acyclic) の数学的証明・検証。
3. **トポロジカル実行スケジューラ (Task Scheduler):** Kahn のアルゴリズムに基づき、依存度が満たされた (In-Degree = 0) タスクから順次実行キューへ投入・処理するエンジン。
4. **C-FFI / Rust バインディング層 (Language Binding):** C 言語互換 ABI ('extern "C"') を介して、Rust などの高レイヤー言語から DAG の構築・検証・実行を行うインターフェース。

2 機能別コアコードの解説

2.1 機能 1: グラフ構築モジュール (Graph Builder)

ノードの登録、有向エッジ ($A \rightarrow B$) の生成、および Kahn のアルゴリズムに必要な入次数 (In-Degree) を管理する。

Listing 1: グラフ構築モジュール (graph_builder.hpp)

```
#include <iostream>
#include <vector>
#include <string>
#include <unordered_map>
#include <functional>

using TaskCallback = std::function<void()>;

struct TaskNode {
    std::string id;
    TaskCallback action;
};

class NinjaGraph {
public:
    std::unordered_map<std::string, TaskNode> nodes;
    std::unordered_map<std::string, std::vector<std::string>> adj_list; // Forward Edges
    std::unordered_map<std::string, int> in_degree; // 入次数管理

    void add_task(const std::string& id, TaskCallback action) {
        if (nodes.find(id) == nodes.end()) {
            nodes[id] = TaskNode{id, action};
            in_degree[id] = 0;
        }
    }

    // parent -> child (parentが完了しないと child は実行できない)
    void add_dependency(const std::string& parent, const std::string& child) {
        if (nodes.find(parent) == nodes.end()) add_task(parent, nullptr);
        if (nodes.find(child) == nodes.end()) add_task(child, nullptr);

        adj_list[parent].push_back(child);
        in_degree[child]++;
    }
};
```

2.2 機能 2: サイクル検知モジュール (Cycle Detector)

DFS によるノードの 3 状態管理 (0: Unvisited, 1: Visiting, 2: Visited) を行い、循環依存が存在するかを判定する。

Listing 2: サイクル検知アルゴリズム (cycle_detector.cpp)

```
enum class NodeMark { UNVISITED, VISITING, VISITED };

bool dfs_check_cycle(const std::string& u,
                    const std::unordered_map<std::string, std::vector<std::string>>&
adj,
                    std::unordered_map<std::string, NodeMark>& mark) {
    mark[u] = NodeMark::VISITING;

    auto it = adj.find(u);
    if (it != adj.end()) {
        for (const auto& v : it->second) {
            if (mark[v] == NodeMark::VISITING) return true; // 循環を発見
            if (mark[v] == NodeMark::UNVISITED) {
                if (dfs_check_cycle(v, adj, mark)) return true;
            }
        }
    }

    mark[u] = NodeMark::VISITED;
    return false;
}

bool contains_cycle(const NinjaGraph& graph) {
    std::unordered_map<std::string, NodeMark> mark;
    for (const auto& pair : graph.nodes) mark[pair.first] = NodeMark::UNVISITED;

    for (const auto& pair : graph.nodes) {
        if (mark[pair.first] == NodeMark::UNVISITED) {
            if (dfs_check_cycle(pair.first, graph.adj_list, mark)) return true;
        }
    }
    return false;
}
```

2.3 機能 3: トポロジカル実行スケジューラ (Task Scheduler)

入次数が 0 となったノードをキュー（またはスレッドプール）に投入し、依存関係を満たした順序でタスクを発火させる。

Listing 3: Kahn のアルゴリズムによるスケジューラ (scheduler.cpp)

```
#include <queue>
#include <stdexcept>

void execute_ninja_dag(NinjaGraph& graph) {
    if (contains_cycle(graph)) {
        throw std::runtime_error("[Fatal] Cyclic dependency detected in DAG.");
    }

    std::queue<std::string> ready_queue;
    auto in_degree_map = graph.in_degree;

    // 初期状態で依存関係なし (In-Degree == 0) のタスクをキューイング
    for (const auto& pair : in_degree_map) {
        if (pair.second == 0) ready_queue.push(pair.first);
    }

    size_t executed_count = 0;

    while (!ready_queue.empty()) {
        std::string current = ready_queue.front();
        ready_queue.pop();
        executed_count++;

        // タスク実行
        if (graph.nodes[current].action) {
            graph.nodes[current].action();
        }

        // 後続タスクの解体と依存度低減
        for (const auto& child : graph.adj_list[current]) {
            in_degree_map[child]--;
            if (in_degree_map[child] == 0) {
                ready_queue.push(child);
            }
        }
    }

    if (executed_count != graph.nodes.size()) {
        throw std::runtime_error("[Error] Unreachable nodes present in graph.");
    }
}
```

3 Rust からの呼び出し機能 (Rust Integration)

C 言語互換 ABI ('extern "C"') を公開エクスポート層として挟むことで、Rust の FFI (Foreign Function Interface) 機能を用いて安全かつ高速に 'ninja' コアエンジン呼び出すことができる。

3.1 機能 4: C-FFI 互換エクスポート層 (C++ Side)

Listing 4: C-FFI エクスポート関数定義 (ninja_ffi.cpp)

```
extern "C" {
    typedef void* NinjaGraphHandle;
    typedef void (*CFunctionCallback)();

    NinjaGraphHandle ninja_graph_create() {
        return reinterpret_cast<NinjaGraphHandle>(new NinjaGraph());
    }

    void ninja_graph_destroy(NinjaGraphHandle handle) {
        delete reinterpret_cast<NinjaGraph*>(handle);
    }

    void ninja_add_task(NinjaGraphHandle handle, const char* id, CFunctionCallback
cb) {
        auto* graph = reinterpret_cast<NinjaGraph*>(handle);
        graph->add_task(std::string(id), [cb]() {
            if (cb) cb();
        });
    }

    void ninja_add_dependency(NinjaGraphHandle handle, const char* parent, const
char* child) {
        auto* graph = reinterpret_cast<NinjaGraph*>(handle);
        graph->add_dependency(std::string(parent), std::string(child));
    }

    int ninja_execute(NinjaGraphHandle handle) {
        auto* graph = reinterpret_cast<NinjaGraph*>(handle);
        try {
            execute_ninja_dag(*graph);
            return 0; // Success
        } catch (...) {
            return -1; // Cycle or Execution Error
        }
    }
}
```

3.2 Rust 側の FFI 宣言と Safe Wrapper の実装

Rust 側では ‘unsafe’ 領域を安全なオブジェクトモデル内にカプセル化する RAII パターンを適用する。

Listing 5: Rust FFI 宣言と Safe ラッパー定義 (ninja.rs)

```
use std::ffi::CString;
use std::os::raw::{c_char, c_int, c_void};

type NinjaGraphHandle = *mut c_void;
type CFunctionCallback = extern "C" fn();

extern "C" {
    fn ninja_graph_create() -> NinjaGraphHandle;
    fn ninja_graph_destroy(handle: NinjaGraphHandle);
    fn ninja_add_task(handle: NinjaGraphHandle, id: *const c_char, cb:
CFunctionCallback);
    fn ninja_add_dependency(handle: NinjaGraphHandle, parent: *const c_char, child:
*const c_char);
    fn ninja_execute(handle: NinjaGraphHandle) -> c_int;
}

// Safe Rust Wrapper
pub struct NinjaEngine {
    handle: NinjaGraphHandle,
}

impl NinjaEngine {
    pub fn new() -> Self {
        let handle = unsafe { ninja_graph_create() };
        NinjaEngine { handle }
    }

    pub fn add_task(&self, id: &str, callback: CFunctionCallback) {
        let c_id = CString::new(id).expect("CString conversion failed");
        unsafe {
            ninja_add_task(self.handle, c_id.as_ptr(), callback);
        }
    }

    pub fn add_dependency(&self, parent: &str, child: &str) {
        let c_parent = CString::new(parent).expect("CString conversion failed");
        let c_child = CString::new(child).expect("CString conversion failed");
        unsafe {
            ninja_add_dependency(self.handle, c_parent.as_ptr(), c_child.as_ptr());
        }
    }

    pub fn execute(&self) -> Result<(), &'static str> {
        let result = unsafe { ninja_execute(self.handle) };
        if result == 0 {
            Ok(())
        }
    }
}
```

```
        } else {
            Err("DAG execution failed due to cycles or internal errors.")
        }
    }
}

impl Drop for NinjaEngine {
    fn drop(&mut self) {
        unsafe {
            ninja_graph_destroy(self.handle);
        }
    }
}
```


3.3 Rust からの DAG 呼出メインコード

以下は、Rust アプリケーションから ‘ninja’ DAG エンジン呼び出し、タスクの依存関係を設定して実行するサンプルプログラムである。

Listing 6: Rust エントリポイント (main.rs)

```
mod ninja;
use ninja::NinjaEngine;

// コールバック関数の定義 (C-ABI 互換)
extern "C" fn task_init() {
    println!("[Rust Callback] Task 1: Environment Initialized.");
}

extern "C" fn task_compile() {
    println!("[Rust Callback] Task 2: Core Engine Compiled.");
}

extern "C" fn task_deploy() {
    println!("[Rust Callback] Task 3: Artifacts Deployed Successfully.");
}

fn main() {
    println!("=== Rust Ninja DAG Integration ===");

    let engine = NinjaEngine::new();

    // 1. タスクの登録
    engine.add_task("Init", task_init);
    engine.add_task("Compile", task_compile);
    engine.add_task("Deploy", task_deploy);

    // 2. 依存関係の設定 (Init -> Compile -> Deploy)
    engine.add_dependency("Init", "Compile");
    engine.add_dependency("Compile", "Deploy");

    // 3. DAG実行
    match engine.execute() {
        Ok(_) => println!("[Success] All Rust-bound DAG tasks executed in topological order."),
        Err(e) => eprintln!("[Error] Execution failed: {}", e),
    }
}
```

4 結論

本ドキュメントでは、‘kyo38/ninja’ リポジトリの DAG エンジンにおける主要な機能ブロック（グラフ構築、DFS サイクル検知、Kahn アルゴリズム実行）のコード仕様を解説し、C-FFI エクスポート層を介した **Rust 言語からのセーフな呼び出し設計** を示した。

この多重構造により、C++コアの持つ高性能なグラフ処理能力と、Rust が持つ高度なメモリ安全性を両立した次世代のタスク依存解析基盤を構築することが可能となる。